

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



BÁO CÁO Project II

**Đề tài: Xây dựng hệ thống
tổng hợp báo cáo cổ phiếu**

Giáo viên hướng dẫn : GV.Ban Hà Bằng
Sinh viên thực hiện : Dương Trần Duy Anh
MSSV : 202416115

Mục lục

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI.....	5
1.1 Lý do chọn đề tài	5
1.2 Yêu cầu bài toán.....	6
1.3 Ý tưởng thực hiện	8
CHƯƠNG 2. CÔNG NGHỆ SỬ DỤNG	9
2.1 PyPDF	9
2.2 Streamlit	10
2.3 SQLite	11
2.4 Google GenAI.....	12
CHƯƠNG 3. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG	13
3.1 Xây dựng giao diện người dùng	13
3.1.1 Mục tiêu thiết kế giao diện	13
3.1.2 Bố cục giao diện	14
3.1.3 Chức năng tải báo cáo.....	15
3.1.4 Chức năng xóa dữ liệu	15
3.1.5 Hiển thị dữ liệu định giá	16
3.2 Xử lý và lưu trữ dữ liệu	16
3.2.1 Kiến trúc xử lý	16
3.2.2 Quy trình xử lý báo cáo	17
3.2.3 Thiết kế cơ sở dữ liệu.....	17
3.2.4 Tính toán Upside	18
3.2.5 Gọi API Gemini và xử lý bằng mô hình ngôn ngữ lớn	18
3.2.6 Prompt Engineering.....	19
3.2.7 Sử dụng JSON Schema để chuẩn hóa dữ liệu	20
3.2.8 Xử lý dữ liệu trùng lặp	21
3.3 Xử lý truy vấn của người dùng	22
3.4 Đánh giá hệ thống	23

3.4.1 Kết quả đạt được.....	23
3.4.2 Đánh giá các chức năng chính	23
3.4.3 So sánh với phương pháp thủ công.....	25
3.4.4 Ưu điểm của hệ thống	26
3.4.5 Hạn chế	26
3.4.6 Khả năng mở rộng	26
CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	27
4.1 Kết luận	27
4.2 Hướng phát triển	28

LỜI NÓI ĐẦU

Trong những năm gần đây, sự phát triển mạnh mẽ của Trí tuệ nhân tạo (Artificial Intelligence - AI), đặc biệt là các mô hình ngôn ngữ lớn (Large Language Models - LLM), đã tạo ra những bước tiến đáng kể trong lĩnh vực xử lý ngôn ngữ tự nhiên và khai phá tri thức từ dữ liệu phi cấu trúc. Các mô hình này không chỉ có khả năng sinh văn bản mà còn hỗ trợ trích xuất thông tin, tóm tắt tài liệu, phân loại nội dung và trả lời câu hỏi với độ chính xác ngày càng cao. Nhờ đó, AI đang dần trở thành một công cụ hỗ trợ đắc lực trong nhiều lĩnh vực như giáo dục, y tế, tài chính, thương mại điện tử và quản trị doanh nghiệp.

Trong lĩnh vực tài chính, đặc biệt là thị trường chứng khoán, thông tin đóng vai trò quyết định đối với chất lượng của các quyết định đầu tư. Mỗi ngày, các công ty chứng khoán phát hành hàng chục báo cáo phân tích doanh nghiệp nhằm đánh giá tình hình hoạt động kinh doanh, triển vọng tăng trưởng, định giá cổ phiếu và đưa ra các khuyến nghị đầu tư. Những báo cáo này thường được xây dựng bởi các chuyên gia phân tích tài chính có nhiều kinh nghiệm và chứa đựng lượng kiến thức chuyên sâu về doanh nghiệp cũng như bối cảnh kinh tế vĩ mô.

Tuy nhiên, phần lớn các báo cáo đều được phát hành dưới dạng tệp PDF với nội dung dài từ vài chục đến hàng trăm trang. Việc đọc và tổng hợp thông tin từ nhiều báo cáo khác nhau đòi hỏi người sử dụng phải dành rất nhiều thời gian để tìm kiếm các chỉ tiêu quan trọng như giá mục tiêu, khuyến nghị đầu tư, tình hình hoạt động doanh nghiệp, dự báo tài chính hay các yếu tố rủi ro. Đối với các nhà đầu tư cá nhân, đây là một công việc tương đối khó khăn bởi lượng thông tin lớn, cách trình bày không đồng nhất giữa các công ty chứng khoán và sự xuất hiện của nhiều thuật ngữ chuyên ngành.

Bên cạnh đó, khi một doanh nghiệp được nhiều công ty chứng khoán cùng phân tích, người dùng thường phải mở đồng thời nhiều tệp PDF để so sánh sự khác biệt trong quan điểm đánh giá. Quá trình này không chỉ mất thời gian mà còn dễ dẫn đến bỏ sót các thông tin quan trọng hoặc đưa ra quyết định dựa trên những nhận định chưa đầy đủ.

Sự phát triển của các mô hình ngôn ngữ lớn mở ra một hướng tiếp cận mới cho bài toán này. Thay vì chỉ lưu trữ nguyên bản các báo cáo, hệ thống có thể tự động đọc toàn bộ nội dung của từng tài liệu, trích xuất những thông tin cần thiết theo một

cấu trúc thống nhất, sau đó lưu trữ vào cơ sở dữ liệu để phục vụ việc tra cứu và so sánh. Điều này giúp giảm đáng kể thời gian xử lý tài liệu, đồng thời nâng cao khả năng khai thác tri thức từ các báo cáo phân tích.

Xuất phát từ nhu cầu thực tế đó, đề tài "**Xây dựng hệ thống tổng hợp báo cáo cổ phiếu**" được thực hiện với mục tiêu xây dựng một ứng dụng có khả năng tự động xử lý các báo cáo chứng khoán dưới dạng PDF, sử dụng mô hình Gemini để phân tích nội dung và trích xuất các thông tin quan trọng dưới định dạng JSON. Dữ liệu sau khi được chuẩn hóa sẽ được lưu trữ trong cơ sở dữ liệu SQLite, đồng thời cung cấp giao diện trực quan thông qua nền tảng Streamlit để người dùng dễ dàng tra cứu, so sánh và tổng hợp các báo cáo của cùng một mã cổ phiếu.

Hệ thống được xây dựng theo hướng đơn giản nhưng có tính mở rộng cao. Quá trình xử lý được chia thành nhiều mô-đun độc lập bao gồm trích xuất văn bản từ PDF, phân tích nội dung bằng AI, lưu trữ dữ liệu, loại bỏ dữ liệu trùng lặp và truy vấn thông tin theo yêu cầu của người dùng. Kiến trúc này giúp việc bảo trì, nâng cấp hoặc thay thế từng thành phần trở nên thuận tiện hơn trong tương lai.

Mặc dù phạm vi của đề tài chủ yếu tập trung vào việc tổng hợp và chuẩn hóa dữ liệu từ các báo cáo chứng khoán, hệ thống vẫn có thể được mở rộng để hỗ trợ nhiều loại tài liệu tài chính khác như báo cáo ngành, báo cáo chiến lược đầu tư hoặc báo cáo kinh tế vĩ mô. Đồng thời, cơ sở dữ liệu được xây dựng cũng có thể trở thành nền tảng cho các hệ thống phân tích tài chính hoặc trợ lý đầu tư thông minh trong tương lai.

Thông qua quá trình thực hiện đề tài, sinh viên có cơ hội vận dụng các kiến thức đã học về lập trình Python, phát triển ứng dụng Web, thiết kế cơ sở dữ liệu, xử lý tài liệu PDF, tích hợp API của mô hình ngôn ngữ lớn và xây dựng hệ thống phần mềm hoàn chỉnh. Đây cũng là cơ hội để tiếp cận các công nghệ AI hiện đại đang được ứng dụng rộng rãi trong thực tiễn, đồng thời rèn luyện kỹ năng thiết kế, triển khai và đánh giá một hệ thống phần mềm theo quy trình phát triển phần mềm hiện đại.

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1 Lý do chọn đề tài

Thị trường chứng khoán là một trong những kênh huy động vốn quan trọng của nền kinh tế, đồng thời cũng là môi trường đầu tư thu hút sự quan tâm của đông đảo nhà đầu tư cá nhân và tổ chức. Trong những năm gần đây, cùng với sự phát triển của thị trường chứng khoán Việt Nam, số lượng các báo cáo phân tích doanh nghiệp được công bố bởi các công ty chứng khoán ngày càng tăng. Những báo cáo này thường bao gồm các nội dung như tình hình hoạt động kinh doanh, phân tích ngành, dự báo kết quả kinh doanh, mô hình định giá doanh nghiệp, đánh giá rủi ro cũng như khuyến nghị đầu tư.

Đối với các tổ chức tài chính, quỹ đầu tư hoặc nhà đầu tư chuyên nghiệp, các báo cáo phân tích là nguồn thông tin có giá trị cao vì được thực hiện bởi đội ngũ chuyên gia tài chính giàu kinh nghiệm. Tuy nhiên, việc khai thác hiệu quả nguồn dữ liệu này vẫn còn gặp nhiều khó khăn do các báo cáo được phát hành dưới dạng PDF, có cấu trúc trình bày không đồng nhất và chứa lượng thông tin rất lớn.

Một doanh nghiệp niêm yết thường được nhiều công ty chứng khoán cùng nghiên cứu. Mỗi đơn vị có thể sử dụng mô hình định giá, giả định tăng trưởng và quan điểm đầu tư khác nhau, dẫn đến sự khác biệt về giá mục tiêu cũng như khuyến nghị. Để có được góc nhìn toàn diện, nhà đầu tư phải đọc đồng thời nhiều báo cáo, đối chiếu các giả định, so sánh các chỉ tiêu tài chính và tổng hợp lại các nhận định quan trọng. Đây là công việc tiêu tốn nhiều thời gian và công sức, đặc biệt đối với nhà đầu tư cá nhân.

Một khó khăn khác là dữ liệu trong các báo cáo chủ yếu tồn tại dưới dạng văn bản phi cấu trúc (Unstructured Data). Máy tính không thể trực tiếp hiểu được nội dung của các đoạn văn để thực hiện việc tìm kiếm, thống kê hoặc so sánh. Nếu muốn xây dựng cơ sở dữ liệu phục vụ phân tích, người dùng thường phải nhập liệu thủ công, vừa mất thời gian vừa dễ xảy ra sai sót.

Sự xuất hiện của các mô hình ngôn ngữ lớn (Large Language Models - LLM) như Gemini đã tạo ra hướng tiếp cận mới cho bài toán xử lý tài liệu. Khác với các phương pháp truyền thống dựa trên biểu thức chính quy hoặc từ khóa cố định, LLM có khả năng hiểu ngữ nghĩa của văn bản, nhận diện thông tin trong nhiều cách diễn đạt khác nhau và sinh kết quả theo cấu trúc được yêu cầu. Điều này đặc

biệt phù hợp với các báo cáo tài chính, nơi cách trình bày và thuật ngữ có thể thay đổi giữa các công ty chứng khoán.

Xuất phát từ những vấn đề trên, đề tài "**Xây dựng hệ thống tổng hợp báo cáo cổ phiếu**" được lựa chọn nhằm nghiên cứu việc kết hợp giữa xử lý tài liệu PDF, mô hình ngôn ngữ lớn Gemini và cơ sở dữ liệu SQLite để xây dựng một hệ thống có khả năng tự động hóa quá trình đọc, phân tích và lưu trữ thông tin từ các báo cáo chứng khoán.

Không giống những ứng dụng chỉ thực hiện tóm tắt văn bản, hệ thống hướng tới việc chuẩn hóa dữ liệu theo cùng một cấu trúc, cho phép người dùng nhanh chóng tra cứu các chỉ tiêu định giá, so sánh khuyến nghị giữa nhiều công ty chứng khoán và xem tổng hợp các luận điểm đầu tư mà không cần đọc toàn bộ tài liệu gốc.

Ngoài giá trị ứng dụng thực tế, đề tài còn giúp sinh viên tiếp cận nhiều công nghệ hiện đại như xử lý tài liệu PDF, Prompt Engineering, Google Gemini API, Streamlit và SQLite. Đây đều là những công nghệ đang được sử dụng rộng rãi trong các hệ thống ứng dụng AI hiện nay.

1.2 Yêu cầu bài toán

Từ thực trạng trên, bài toán đặt ra là xây dựng một hệ thống có khả năng tiếp nhận các báo cáo chứng khoán dưới dạng PDF, tự động phân tích nội dung, trích xuất các thông tin quan trọng và lưu trữ vào cơ sở dữ liệu để phục vụ việc tra cứu sau này.

Khác với các bài toán OCR hoặc tìm kiếm văn bản thông thường, hệ thống không chỉ cần đọc được nội dung của tài liệu mà còn phải hiểu ý nghĩa của từng phần trong báo cáo. Ví dụ, hệ thống phải xác định được đâu là giá mục tiêu, đâu là mức giá tại thời điểm phát hành báo cáo, đâu là khuyến nghị đầu tư cũng như các nhận định liên quan đến triển vọng kinh doanh hoặc các yếu tố rủi ro của doanh nghiệp.

Một yêu cầu quan trọng khác là dữ liệu đầu ra phải được chuẩn hóa theo cùng một định dạng. Trong đề tài này, tất cả kết quả trả về từ Gemini đều được yêu cầu dưới dạng JSON với các trường dữ liệu xác định trước như:

- Mã cổ phiếu (Ticker)
- Công ty chứng khoán phát hành báo cáo
- Ngày phát hành

- Khuyến nghị đầu tư
- Giá cổ phiếu tại thời điểm phát hành
- Giá mục tiêu
- Phân tích tình hình doanh nghiệp
- Triển vọng kinh doanh
- Dự báo tài chính
- Các yếu tố rủi ro
- Luận điểm đầu tư

Việc chuẩn hóa dữ liệu giúp toàn bộ các báo cáo, mặc dù được viết bởi các công ty chứng khoán khác nhau, đều có thể được lưu trữ trong cùng một cấu trúc cơ sở dữ liệu và dễ dàng truy vấn sau này.

Ngoài ra, hệ thống cần giải quyết bài toán trùng lặp dữ liệu. Trong thực tế, người dùng có thể vô tình tải cùng một báo cáo nhiều lần hoặc tải lại các báo cáo đã được xử lý trước đó. Nếu không có cơ chế kiểm tra, cơ sở dữ liệu sẽ xuất hiện nhiều bản ghi giống nhau, gây lãng phí bộ nhớ và làm sai lệch kết quả thống kê.

Một yêu cầu khác là hệ thống phải cung cấp giao diện trực quan, đơn giản và dễ sử dụng. Người dùng không cần thao tác với cơ sở dữ liệu mà chỉ cần tải tệp PDF lên hệ thống, sau đó nhập mã cổ phiếu để xem toàn bộ thông tin đã được tổng hợp.

Bên cạnh các yêu cầu chức năng, hệ thống cũng cần đáp ứng một số yêu cầu phi chức năng như:

- Thời gian xử lý mỗi báo cáo ở mức chấp nhận được.
- Dữ liệu được lưu trữ ổn định sau khi xử lý.
- Có khả năng mở rộng để bổ sung thêm các trường dữ liệu mới.
- Giao diện thân thiện với người sử dụng.
- Dễ bảo trì và nâng cấp.

Những yêu cầu trên là cơ sở để thiết kế kiến trúc và lựa chọn công nghệ cho hệ thống.

1.3 Ý tưởng thực hiện

Ý tưởng cốt lõi của hệ thống là chuyển đổi các báo cáo chứng khoán từ dữ liệu phi cấu trúc thành dữ liệu có cấu trúc bằng cách kết hợp xử lý PDF và mô hình ngôn ngữ lớn Gemini.

Toàn bộ quá trình hoạt động của hệ thống được chia thành năm giai đoạn chính.

Giai đoạn 1. Trích xuất văn bản từ PDF

Khi người dùng tải báo cáo lên hệ thống, thư viện **PyPDF** sẽ đọc từng trang của tài liệu và trích xuất toàn bộ phần văn bản. Kết quả thu được là một chuỗi văn bản liên tục chứa đầy đủ nội dung của báo cáo.

Việc sử dụng PyPDF giúp hệ thống xử lý được phần lớn các báo cáo chứng khoán hiện nay mà không cần chuyển đổi sang định dạng khác.

Giai đoạn 2. Phân tích bằng Gemini

Toàn bộ văn bản sau khi được trích xuất sẽ được gửi đến mô hình Gemini thông qua Google GenAI SDK.

Thay vì yêu cầu AI tóm tắt ngắn gọn, hệ thống sử dụng Prompt Engineering để yêu cầu mô hình:

- đọc toàn bộ báo cáo;
- xác định các thông tin quan trọng;
- phân tích chuyên sâu từng nội dung;
- trả kết quả dưới đúng định dạng JSON đã thiết kế.

Việc quy định rõ kiểu dữ liệu của từng trường trong JSON Schema giúp hạn chế lỗi sinh dữ liệu không đúng định dạng và tăng tính ổn định khi lưu vào cơ sở dữ liệu.

Giai đoạn 3. Lưu trữ dữ liệu

Sau khi nhận được kết quả từ Gemini, hệ thống kiểm tra tính hợp lệ của dữ liệu trước khi ghi vào SQLite.

Trong quá trình lưu trữ, hệ thống đồng thời kiểm tra xem báo cáo đã tồn tại hay chưa thông qua bộ ba thông tin:

- mã cổ phiếu,
- công ty chứng khoán,
- ngày phát hành.

Nếu dữ liệu đã tồn tại, hệ thống sẽ bỏ qua thay vì ghi thêm bản ghi mới.

Giai đoạn 4. Truy vấn dữ liệu

Khi người dùng nhập mã cổ phiếu, hệ thống sẽ truy vấn toàn bộ các báo cáo liên quan từ cơ sở dữ liệu.

Các dữ liệu định giá như giá hiện tại, giá mục tiêu và tỷ lệ Upside được tổng hợp thành bảng để người dùng dễ dàng so sánh giữa các công ty chứng khoán.

Đồng thời, các nội dung phân tích chuyên sâu như triển vọng kinh doanh, rủi ro hay luận điểm đầu tư cũng được hiển thị để người dùng có cái nhìn tổng quan về doanh nghiệp.

Giai đoạn 5. Hiển thị giao diện

Toàn bộ chức năng của hệ thống được triển khai trên nền tảng Streamlit.

Giao diện được thiết kế theo hướng đơn giản, tập trung vào ba chức năng chính:

- tải báo cáo PDF;
- tra cứu theo mã cổ phiếu;
- xóa các báo cáo không còn cần thiết.

Nhờ kiến trúc module hóa, các thành phần xử lý PDF, AI, cơ sở dữ liệu và giao diện được tách biệt với nhau, giúp hệ thống dễ bảo trì và có thể mở rộng thêm các chức năng trong tương lai như hỗ trợ nhiều loại tài liệu tài chính, tìm kiếm theo khoảng thời gian hoặc tích hợp nhiều mô hình AI khác nhau.

CHƯƠNG 2. CÔNG NGHỆ SỬ DỤNG

2.1 PyPDF

Một trong những bước quan trọng nhất của hệ thống là chuyển đổi báo cáo chứng khoán từ định dạng PDF sang văn bản thuần (Plain Text). Đây là điều kiện tiên quyết để mô hình ngôn ngữ lớn có thể đọc và phân tích nội dung tài liệu. Để thực

hiện nhiệm vụ này, hệ thống sử dụng thư viện PyPDF, một thư viện mã nguồn mở phổ biến trong hệ sinh thái Python dành cho việc xử lý các tệp PDF.

PyPDF hỗ trợ nhiều chức năng như đọc tài liệu, trích xuất văn bản, tách hoặc ghép các tệp PDF, xoay trang và truy cập các thuộc tính của tài liệu. Trong phạm vi của đề tài, chức năng được sử dụng chủ yếu là đọc nội dung văn bản trên từng trang của báo cáo.

Quá trình trích xuất được thực hiện thông qua lớp PdfReader. Sau khi người dùng tải báo cáo lên hệ thống, tệp PDF được truyền vào hàm `extract_text_from_pdf()`. Hàm này lần lượt duyệt qua tất cả các trang trong tài liệu và sử dụng phương thức `extract_text()` để lấy nội dung văn bản. Các đoạn văn bản của từng trang sau đó được nối lại thành một chuỗi duy nhất để phục vụ cho bước phân tích bằng Gemini.

Việc sử dụng vòng lặp để xử lý từng trang giúp hệ thống có thể đọc được các báo cáo có số lượng trang khác nhau mà không cần cấu hình trước. Đồng thời, chương trình cũng sử dụng cơ chế bắt ngoại lệ (`try...except`) nhằm bỏ qua các trang không thể trích xuất được nội dung thay vì làm gián đoạn toàn bộ quá trình xử lý. Điều này giúp tăng tính ổn định của hệ thống khi gặp các tệp PDF có cấu trúc phức tạp hoặc chứa nhiều thành phần đồ họa.

Một ưu điểm quan trọng của PyPDF là không phụ thuộc vào các phần mềm bên ngoài như Adobe Acrobat và hoàn toàn hoạt động bằng Python. Điều này giúp việc triển khai hệ thống trở nên đơn giản hơn và dễ dàng chạy trên nhiều hệ điều hành khác nhau.

Tuy nhiên, PyPDF cũng tồn tại một số hạn chế. Thư viện chỉ hoạt động tốt với các báo cáo chứa văn bản số (Digital PDF). Đối với các báo cáo được lưu dưới dạng hình ảnh quét (Scanned PDF), PyPDF không thể nhận dạng ký tự và cần kết hợp thêm công nghệ OCR như Tesseract hoặc Google Vision AI. Trong phạm vi của đề tài, các báo cáo của các công ty chứng khoán đều là PDF văn bản nên hạn chế này không ảnh hưởng đến kết quả thực nghiệm.

2.2 Streamlit

Sau khi hoàn thành các mô-đun xử lý dữ liệu, hệ thống cần một giao diện trực quan để người dùng có thể thao tác mà không cần sử dụng dòng lệnh. Thay vì xây dựng

một ứng dụng Web hoàn chỉnh bằng HTML, CSS và JavaScript, đề tài lựa chọn Streamlit để phát triển giao diện người dùng.

Streamlit là một framework mã nguồn mở cho phép xây dựng các ứng dụng Web bằng Python. Điểm mạnh lớn nhất của Streamlit là lập trình viên chỉ cần viết mã Python mà không phải xây dựng riêng phần Front-end. Điều này giúp rút ngắn đáng kể thời gian phát triển, đặc biệt phù hợp với các dự án nghiên cứu hoặc nguyên mẫu (Prototype).

Trong hệ thống, Streamlit đảm nhiệm toàn bộ việc hiển thị giao diện và tương tác với người dùng. Ngay khi chương trình khởi động, hàm `st.set_page_config()` được sử dụng để cấu hình tiêu đề và bố cục của trang web. Sau đó, các thành phần giao diện như tiêu đề, thanh bên (Sidebar), vùng nhập dữ liệu và bảng kết quả được tạo thông qua các hàm có sẵn của Streamlit.

Thanh Sidebar được sử dụng để chứa nhóm chức năng xử lý báo cáo và xóa dữ liệu. Người dùng có thể tải lên đồng thời nhiều tệp PDF bằng thành phần `st.file_uploader()`. Sau khi nhấn nút "Lưu trữ", hệ thống sẽ lần lượt xử lý từng báo cáo, hiển thị tiến trình bằng `st.spinner()` và thông báo kết quả bằng `st.success()` hoặc `st.error()`. Cách tổ chức này giúp người dùng dễ dàng theo dõi trạng thái của từng báo cáo trong quá trình xử lý.

Đối với chức năng tra cứu, Streamlit cung cấp ô nhập liệu `st.text_input()` để người dùng nhập mã cổ phiếu. Khi có dữ liệu, chương trình sẽ truy vấn cơ sở dữ liệu và hiển thị kết quả dưới dạng bảng thông qua thư viện Pandas. Ngoài các thông tin cơ bản như công ty chứng khoán, ngày phát hành hay khuyến nghị đầu tư, hệ thống còn tự động tính toán tỷ lệ Upside (%) và sử dụng định dạng màu sắc để giúp người dùng dễ dàng nhận biết các mức tăng hoặc giảm kỳ vọng.

Việc lựa chọn Streamlit giúp giảm đáng kể khối lượng công việc so với việc xây dựng một hệ thống Web truyền thống. Đồng thời, giao diện vẫn đáp ứng tốt các yêu cầu của đề tài như tải báo cáo, quản lý dữ liệu và tra cứu thông tin.

2.3 SQLite

Sau khi thông tin được Gemini phân tích và chuẩn hóa, dữ liệu cần được lưu trữ lâu dài để phục vụ cho việc tra cứu và so sánh giữa nhiều báo cáo khác nhau. Đối với đề tài này, hệ quản trị cơ sở dữ liệu được lựa chọn là SQLite.

SQLite là hệ quản trị cơ sở dữ liệu quan hệ có kích thước nhỏ gọn, không yêu cầu cài đặt máy chủ riêng và toàn bộ dữ liệu được lưu trong một tệp duy nhất. Điều này giúp việc triển khai hệ thống trở nên đơn giản hơn rất nhiều so với các hệ quản trị như MySQL hay PostgreSQL.

Khi chương trình khởi động, hàm `init_db()` sẽ tự động kiểm tra sự tồn tại của cơ sở dữ liệu. Nếu bảng `reports` chưa được tạo, chương trình sẽ sinh bảng mới với đầy đủ các trường dữ liệu cần thiết như mã cổ phiếu, công ty chứng khoán, ngày phát hành, giá hiện tại, giá mục tiêu, khuyến nghị đầu tư, tình hình doanh nghiệp, triển vọng kinh doanh, dự báo tài chính, rủi ro và luận điểm đầu tư.

Để tránh việc lưu trùng báo cáo, trước khi thực hiện lệnh `INSERT`, hệ thống tiến hành truy vấn kiểm tra xem đã tồn tại bản ghi có cùng mã cổ phiếu, công ty chứng khoán và ngày phát hành hay chưa. Nếu dữ liệu đã tồn tại, chương trình sẽ bỏ qua thao tác ghi dữ liệu. Cơ chế này giúp cơ sở dữ liệu luôn nhất quán và tránh phát sinh các bản ghi giống nhau.

Đối với một hệ thống có quy mô vừa và nhỏ như đề tài này, SQLite đáp ứng đầy đủ các yêu cầu về tốc độ, tính ổn định và khả năng triển khai. Trong trường hợp hệ thống được mở rộng cho nhiều người dùng truy cập đồng thời, SQLite có thể được thay thế bằng PostgreSQL hoặc MySQL mà không cần thay đổi nhiều ở tầng ứng dụng.

2.4 Google GenAI

Thành phần quan trọng nhất của hệ thống là mô hình ngôn ngữ lớn Gemini do Google phát triển. Khác với các phương pháp xử lý văn bản truyền thống dựa trên từ khóa hoặc biểu thức chính quy, Gemini có khả năng hiểu ngữ cảnh, suy luận nội dung và trích xuất thông tin theo yêu cầu của người sử dụng.

Trong hệ thống, sau khi toàn bộ nội dung PDF được chuyển thành văn bản, chuỗi văn bản này sẽ được đưa vào một Prompt được thiết kế sẵn. Prompt yêu cầu Gemini đóng vai trò là chuyên gia phân tích tài chính, đọc toàn bộ báo cáo, trích xuất các thông tin quan trọng và trả kết quả theo định dạng JSON với các trường dữ liệu đã định nghĩa trước.

Một điểm đáng chú ý trong quá trình triển khai là việc sử dụng JSON Schema để ràng buộc định dạng đầu ra. Thay vì để mô hình sinh văn bản tự do, mỗi trường dữ

liệu như ticker, source, recommendation, current_price_at_report hay target_price đều được khai báo rõ kiểu dữ liệu. Điều này giúp giảm đáng kể các lỗi định dạng và giúp dữ liệu có thể được ghi trực tiếp vào cơ sở dữ liệu mà không cần xử lý thủ công.

Ngoài việc trích xuất các thông tin định lượng, Gemini còn được yêu cầu viết các phân tích chuyên sâu về tình hình doanh nghiệp, triển vọng kinh doanh, dự báo tài chính, các yếu tố rủi ro và luận điểm đầu tư. Các nội dung này không chỉ được tóm tắt mà còn được diễn giải thành các đoạn văn hoàn chỉnh, giúp người dùng có thể nhanh chóng nắm bắt nội dung chính của báo cáo mà không cần đọc toàn bộ tài liệu gốc.

Việc tích hợp Gemini giúp hệ thống vượt qua hạn chế của các phương pháp trích xuất truyền thống. Thay vì phải xây dựng nhiều quy tắc riêng cho từng mẫu báo cáo, hệ thống có thể xử lý linh hoạt các tài liệu được trình bày theo nhiều định dạng khác nhau, miễn là nội dung vẫn chứa các thông tin cần thiết.

CHƯƠNG 3. THIẾT KẾ, TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

3.1 Xây dựng giao diện người dùng

3.1.1 Mục tiêu thiết kế giao diện

Một hệ thống xử lý dữ liệu bằng trí tuệ nhân tạo sẽ khó phát huy hiệu quả nếu người sử dụng phải thao tác thông qua dòng lệnh hoặc chỉnh sửa trực tiếp mã nguồn. Vì vậy, giao diện người dùng (User Interface - UI) đóng vai trò là cầu nối giữa người dùng và các mô-đun xử lý phía sau.

Đối với đề tài này, giao diện được thiết kế với tiêu chí đơn giản, trực quan và dễ sử dụng, phù hợp với cả những người không có kiến thức chuyên sâu về lập trình. Người dùng chỉ cần tải báo cáo PDF lên hệ thống, chờ quá trình xử lý hoàn tất và nhập mã cổ phiếu để xem kết quả.

Toàn bộ giao diện được phát triển bằng Streamlit. Đây là framework cho phép xây dựng ứng dụng Web chỉ bằng Python mà không cần viết mã HTML, CSS hay JavaScript. Nhờ vậy, thời gian phát triển được rút ngắn đáng kể nhưng vẫn đảm bảo khả năng tương tác tốt.

Ngay khi chương trình khởi động, hệ thống thiết lập tiêu đề cửa sổ và bố cục dạng rộng (wide) nhằm tận dụng tối đa không gian hiển thị trên màn hình. Đồng thời, cơ sở dữ liệu cũng được khởi tạo nếu chưa tồn tại để đảm bảo các chức năng phía sau có thể hoạt động bình thường.

3.1.2 Bố cục giao diện

Giao diện chính của hệ thống được chia thành hai khu vực.

Thanh Sidebar

Thanh Sidebar nằm bên trái màn hình, bao gồm toàn bộ các chức năng quản trị dữ liệu.

Các chức năng chính bao gồm:

- tải lên nhiều báo cáo PDF cùng lúc;
- lưu dữ liệu sau khi AI xử lý;
- xóa các báo cáo đã lưu trong cơ sở dữ liệu.

Việc nhóm các chức năng quản trị vào Sidebar giúp khu vực chính của giao diện chỉ tập trung hiển thị kết quả tra cứu, tạo cảm giác trực quan hơn đối với người sử dụng.

Khu vực tra cứu

Ở phần trung tâm màn hình, hệ thống cung cấp một ô nhập mã cổ phiếu.

Khi người dùng nhập ví dụ:

FPT

Hoặc

HPG

hệ thống sẽ tự động chuyển mã cổ phiếu sang chữ in hoa trước khi gửi truy vấn xuống cơ sở dữ liệu nhằm tránh sai lệch do khác biệt giữa chữ hoa và chữ thường.

Sau khi có kết quả, hệ thống sẽ hiển thị hai nhóm thông tin:

- Bảng định giá.
- Các phân tích chuyên sâu.

Cách bố trí này giúp người dùng vừa có thể quan sát nhanh các số liệu định lượng, vừa đọc được các nhận định định tính của từng công ty chứng khoán.

3.1.3 Chức năng tải báo cáo

Đây là chức năng quan trọng nhất của giao diện.

Streamlit hỗ trợ thành phần `file_uploader()` cho phép tải đồng thời nhiều tệp PDF.

Khi người dùng nhấn nút Lưu trữ, chương trình sẽ lần lượt xử lý từng báo cáo theo quy trình sau:

1. Đọc PDF.
2. Trích xuất văn bản.
3. Gửi nội dung sang Gemini.
4. Nhận dữ liệu JSON.
5. Ghi dữ liệu vào SQLite.

Trong thời gian xử lý, Streamlit sử dụng `spinner()` để hiển thị trạng thái đang làm việc. Điều này giúp người dùng biết hệ thống vẫn đang hoạt động, đặc biệt khi xử lý các báo cáo dài từ 30–100 trang.

Sau khi hoàn thành, hệ thống hiển thị thông báo thành công hoặc lỗi tương ứng đối với từng báo cáo.

Việc xử lý lần lượt từng tệp giúp giảm tải cho API Gemini và dễ dàng xác định báo cáo nào xảy ra lỗi nếu có sự cố.

3.1.4 Chức năng xóa dữ liệu

Ngoài việc bổ sung dữ liệu, hệ thống còn hỗ trợ xóa báo cáo khỏi cơ sở dữ liệu.

Người dùng chỉ cần nhập danh sách ID, ví dụ:

3,5,8

Sau khi nhấn nút Xóa báo cáo, chương trình sẽ:

- tách chuỗi theo dấu phẩy;
- chuyển từng phần tử sang số nguyên;

- gọi hàm xóa trong tầng cơ sở dữ liệu;
- tải lại giao diện nếu thao tác thành công.

Việc xóa theo ID thay vì theo mã cổ phiếu giúp người dùng chỉ loại bỏ đúng báo cáo mong muốn mà không ảnh hưởng đến các báo cáo khác của cùng doanh nghiệp.

3.1.5 Hiện thị dữ liệu định giá

Sau khi người dùng nhập mã cổ phiếu, chương trình truy vấn toàn bộ các báo cáo liên quan.

Kết quả được chuyển thành DataFrame của Pandas trước khi hiển thị.

Các cột được trình bày bao gồm:

- ID
- Công ty chứng khoán
- Ngày phát hành
- Khuyến nghị
- Giá tại thời điểm báo cáo
- Giá mục tiêu
- Upside (%)

Trong đó, chỉ tiêu Upside được tính toán tự động từ dữ liệu đã lưu.

Nếu Upside dương, ô dữ liệu sẽ được tô màu xanh.

Nếu Upside âm, hệ thống hiển thị màu đỏ.

Cách trực quan hóa này giúp người dùng nhanh chóng nhận biết quan điểm lạc quan hay bi quan của từng công ty chứng khoán mà không cần tự tính toán.

3.2 Xử lý và lưu trữ dữ liệu

3.2.1 Kiến trúc xử lý

Hệ thống được chia thành ba mô-đun chính:

- Giao diện (Presentation Layer)

- Xử lý AI (Business Logic Layer)
- Cơ sở dữ liệu (Data Layer)

Ba tầng này hoạt động độc lập với nhau.

Điều này mang lại nhiều lợi ích:

- dễ bảo trì;
- dễ kiểm thử;
- có thể thay thế từng thành phần mà không ảnh hưởng toàn bộ hệ thống.

Ví dụ:

Nếu trong tương lai muốn thay Gemini bằng GPT hoặc Claude thì chỉ cần sửa mô-đun `ai_processor.py`, toàn bộ phần giao diện và cơ sở dữ liệu vẫn giữ nguyên.

3.2.2 Quy trình xử lý báo cáo

Khi người dùng tải lên một báo cáo PDF, hệ thống sẽ thực hiện chuỗi xử lý sau:

PDF => PyPDF => Text => Gemini => JSON => SQLite => Tra cứu

Quá trình này hoàn toàn tự động. Người dùng không cần nhập bất kỳ thông tin nào ngoài việc tải tệp PDF. Đây là điểm khác biệt so với nhiều hệ thống lưu trữ tài liệu truyền thống vốn yêu cầu nhập thủ công các thông tin như mã cổ phiếu hoặc công ty chứng khoán.

3.2.3 Thiết kế cơ sở dữ liệu

Toàn bộ dữ liệu được lưu trong bảng `reports`.

Bảng này bao gồm các trường:

Tên trường	Ý nghĩa
<code>id</code>	Khóa chính
<code>ticker</code>	Mã cổ phiếu
<code>source</code>	Công ty chứng khoán
<code>date</code>	Ngày phát hành
<code>recommendation</code>	Khuyến nghị
<code>current_price_at_report</code>	Giá tại thời điểm báo cáo
<code>target_price</code>	Giá mục tiêu
<code>business_status</code>	Phân tích doanh nghiệp
<code>business_outlook</code>	Triển vọng

financial forecast	Dự báo tài chính
risks	Các yếu tố rủi ro
investment thesis	Luận điểm đầu tư
created_at	Thời gian lưu

Việc thiết kế bảng theo dạng chuẩn hóa giúp dễ dàng mở rộng trong tương lai.

Ví dụ:

Nếu muốn lưu thêm:

- EPS
- P/E
- ROE
- EV/EBITDA

thì chỉ cần bổ sung thêm các cột tương ứng mà không phải thay đổi cấu trúc của toàn bộ hệ thống.

3.3.4 Tính toán Upside

Một trong những thông tin quan trọng nhất của báo cáo chứng khoán là mức tăng kỳ vọng (Upside).

Thay vì lưu trực tiếp vào cơ sở dữ liệu, hệ thống tính toán động mỗi lần truy vấn.

Công thức sử dụng là:

$$\text{Upside (\%)} = (\text{Target Price} - \text{Current Price}) / \text{Current Price} \times 100$$

Nếu dữ liệu không hợp lệ hoặc bằng 0 thì Upside sẽ được gán bằng 0 nhằm tránh lỗi chia cho 0.

Cách tiếp cận này giúp dữ liệu luôn nhất quán ngay cả khi giá hoặc mục tiêu được chỉnh sửa sau này.

3.2.5 Gọi API Gemini và xử lý bằng mô hình ngôn ngữ lớn

Sau khi văn bản được trích xuất từ tệp PDF, hệ thống chuyển sang giai đoạn quan trọng nhất là phân tích nội dung bằng mô hình ngôn ngữ lớn Gemini của Google. Đây là thành phần quyết định khả năng "hiểu" báo cáo thay vì chỉ đọc được văn bản.

Trong mô-đun `ai_processor.py`, hệ thống khởi tạo đối tượng Client của Google GenAI SDK thông qua khóa API đã được cấu hình. Sau khi nhận được chuỗi văn bản từ báo cáo PDF, chương trình xây dựng một Prompt hoàn chỉnh rồi gửi đến Gemini để xử lý.

Khác với các phương pháp truyền thống sử dụng biểu thức chính quy (Regular Expression) hoặc tìm kiếm theo từ khóa, Gemini có khả năng hiểu ngữ nghĩa của văn bản. Điều này đặc biệt quan trọng đối với các báo cáo chứng khoán, bởi cùng một thông tin nhưng mỗi công ty chứng khoán có thể trình bày theo nhiều cách khác nhau.

Ví dụ, giá mục tiêu có thể được ghi là:

- Giá mục tiêu
- Target Price
- Fair Value
- Giá hợp lý
- Giá kỳ vọng
- Giá mục tiêu 12 tháng

Nếu sử dụng phương pháp dò tìm từ khóa cố định, hệ thống sẽ phải xây dựng rất nhiều quy tắc khác nhau và vẫn khó bao quát hết các trường hợp. Trong khi đó, Gemini có khả năng suy luận rằng tất cả các cách diễn đạt trên đều đề cập đến cùng một khái niệm và trích xuất đúng giá trị cần thiết.

Ngoài việc nhận diện các thông tin định lượng, mô hình còn được yêu cầu phân tích những nội dung định tính như triển vọng kinh doanh, rủi ro và luận điểm đầu tư. Đây là những phần chứa nhiều đoạn văn dài, đòi hỏi khả năng hiểu ngữ cảnh mà các phương pháp xử lý văn bản truyền thống rất khó thực hiện.

Việc tích hợp Gemini giúp hệ thống giảm đáng kể khối lượng xử lý thủ công, đồng thời tăng tính linh hoạt khi làm việc với nhiều mẫu báo cáo khác nhau.

3.2.6 Prompt Engineering

Một trong những yếu tố quyết định chất lượng đầu ra của hệ thống là Prompt Engineering. Trong đề tài này, Prompt không chỉ đơn thuần yêu cầu AI tóm tắt báo cáo mà được thiết kế như một tập hợp các chỉ dẫn chi tiết nhằm kiểm soát chặt chẽ nội dung trả về.

Prompt yêu cầu Gemini đóng vai trò là chuyên gia phân tích tài chính, đọc toàn bộ báo cáo và thực hiện đồng thời nhiều nhiệm vụ:

- xác định mã cổ phiếu;
- xác định công ty chứng khoán phát hành báo cáo;
- xác định ngày phát hành;
- trích xuất giá hiện tại và giá mục tiêu;
- xác định khuyến nghị đầu tư;
- phân tích chi tiết tình hình doanh nghiệp;
- phân tích triển vọng kinh doanh;
- tổng hợp dự báo tài chính;
- phân tích các yếu tố rủi ro;
- xây dựng luận điểm đầu tư.

Khác với nhiều ứng dụng chỉ yêu cầu AI sinh vài câu tóm tắt, Prompt của hệ thống còn quy định rõ rằng mỗi nội dung phải được trình bày thành một đoạn phân tích đầy đủ, có luận điểm và dẫn chứng từ chính báo cáo, đồng thời không được suy diễn ngoài dữ liệu gốc.

Việc mô tả chi tiết yêu cầu giúp giảm hiện tượng AI trả lời quá ngắn hoặc thiếu thông tin. Đồng thời, các hướng dẫn như "không suy đoán ngoài nội dung báo cáo" cũng góp phần hạn chế hiện tượng hallucination của mô hình ngôn ngữ lớn.

Trong quá trình thử nghiệm, nhóm nhận thấy rằng chất lượng đầu ra phụ thuộc rất lớn vào cách xây dựng Prompt. Khi Prompt chỉ yêu cầu "tóm tắt báo cáo", kết quả thường thiếu nhiều thông tin quan trọng. Sau khi bổ sung các ràng buộc về vai trò, độ dài và định dạng đầu ra, chất lượng dữ liệu được cải thiện rõ rệt.

3.2.7 Sử dụng JSON Schema để chuẩn hóa dữ liệu

Một vấn đề thường gặp khi làm việc với các mô hình ngôn ngữ lớn là kết quả đầu ra không ổn định. Nếu chỉ yêu cầu AI trả lời bằng văn bản, mô hình có thể thay đổi thứ tự thông tin hoặc sử dụng các tên trường khác nhau giữa các lần sinh dữ liệu.

Để giải quyết vấn đề này, hệ thống sử dụng JSON Schema do Google GenAI SDK cung cấp.

Toàn bộ cấu trúc dữ liệu được khai báo trước với các trường cụ thể như:

- ticker

- source
- date
- recommendation
- current_price_at_report
- target_price
- business_status
- business_outlook
- financial_forecast
- risks
- investment_thesis

Đối với mỗi trường, hệ thống quy định rõ kiểu dữ liệu như chuỗi ký tự (String) hoặc số thực (Number). Ví dụ, hai trường `current_price_at_report` và `target_price` bắt buộc phải có kiểu Number để thuận tiện cho việc tính toán Upside sau này.

Việc sử dụng JSON Schema mang lại nhiều lợi ích:

- giảm lỗi định dạng dữ liệu;
- hạn chế việc AI sinh thêm các trường không cần thiết;
- giúp dữ liệu có thể ghi trực tiếp vào SQLite;
- giảm khối lượng xử lý sau khi nhận kết quả từ AI.

Đây là một trong những kỹ thuật quan trọng giúp tăng tính ổn định của toàn bộ hệ thống.

3.2.8 Xử lý dữ liệu trùng lặp

Trong thực tế, người dùng có thể tải cùng một báo cáo nhiều lần hoặc tải lại báo cáo đã được xử lý trước đó. Nếu hệ thống không có cơ chế kiểm tra, cơ sở dữ liệu sẽ xuất hiện nhiều bản ghi giống nhau, gây lãng phí dung lượng lưu trữ và làm sai lệch kết quả tra cứu.

Để khắc phục vấn đề này, trước khi thực hiện lệnh INSERT, chương trình tiến hành truy vấn kiểm tra sự tồn tại của bản ghi dựa trên ba thuộc tính:

- mã cổ phiếu;
- công ty chứng khoán phát hành;
- ngày phát hành báo cáo.

Nếu truy vấn trả về kết quả, chương trình kết luận rằng báo cáo đã tồn tại và bỏ qua thao tác lưu dữ liệu. Ngược lại, nếu chưa tồn tại, dữ liệu mới sẽ được ghi vào cơ sở dữ liệu.

Giải pháp này có ưu điểm là đơn giản, tốc độ xử lý nhanh và phù hợp với quy mô của hệ thống. Đồng thời, việc sử dụng ba thuộc tính trên làm khóa nhận diện giúp phân biệt được các báo cáo khác nhau của cùng một doanh nghiệp nhưng được phát hành ở các thời điểm khác nhau.

3.3 Xử lý truy vấn của người dùng

Sau khi dữ liệu được lưu trữ trong SQLite, hệ thống cung cấp chức năng tra cứu nhằm hỗ trợ người dùng khai thác thông tin một cách nhanh chóng.

Người dùng chỉ cần nhập mã cổ phiếu vào ô tìm kiếm trên giao diện. Chương trình sẽ chuẩn hóa dữ liệu đầu vào bằng cách loại bỏ khoảng trắng và chuyển toàn bộ ký tự sang chữ in hoa trước khi thực hiện truy vấn. Điều này giúp tránh tình trạng cùng một mã cổ phiếu nhưng được nhập dưới nhiều định dạng khác nhau như "fpt", "Fpt" hoặc "FPT".

Sau khi nhận được mã cổ phiếu, hệ thống gọi hai hàm trong tầng cơ sở dữ liệu:

- `get_stock_valuation()`: lấy các thông tin định lượng như giá hiện tại, giá mục tiêu, khuyến nghị đầu tư và tính toán tỷ lệ Upside.
- `get_stock_pillars()`: lấy các thông tin định tính bao gồm tình hình doanh nghiệp, triển vọng kinh doanh, dự báo tài chính, rủi ro và luận điểm đầu tư.

Kết quả định lượng được chuyển thành DataFrame của thư viện Pandas để hiển thị dưới dạng bảng. Việc trình bày dữ liệu theo dạng bảng giúp người dùng dễ dàng so sánh đánh giá của nhiều công ty chứng khoán đối với cùng một mã cổ phiếu.

Đối với các thông tin định tính, hệ thống hiển thị riêng theo từng báo cáo nhằm giữ nguyên ngữ cảnh phân tích của từng công ty chứng khoán. Cách trình bày này giúp người dùng dễ dàng nhận biết những điểm tương đồng và khác biệt trong quan điểm đầu tư giữa các đơn vị phân tích.

Ngoài chức năng tra cứu, hệ thống còn hỗ trợ xóa các báo cáo không còn cần thiết thông qua mã ID. Sau khi thao tác xóa thành công, giao diện được tải lại để cập

nhật dữ liệu mới nhất, giúp người dùng luôn làm việc với cơ sở dữ liệu đã được đồng bộ.

3.4 Đánh giá hệ thống

3.4.1 Kết quả đạt được

Sau quá trình thiết kế và triển khai, hệ thống đã hoàn thành các mục tiêu ban đầu đặt ra. Người dùng có thể tải lên nhiều báo cáo chứng khoán dưới dạng PDF, hệ thống tự động trích xuất nội dung, sử dụng mô hình Gemini để phân tích và chuẩn hóa dữ liệu, sau đó lưu trữ vào cơ sở dữ liệu SQLite. Các thông tin quan trọng như mã cổ phiếu, công ty chứng khoán, giá mục tiêu, giá tại thời điểm phát hành báo cáo, khuyến nghị đầu tư và các nội dung phân tích chuyên sâu đều được lưu lại để phục vụ việc tra cứu sau này.

Quá trình xử lý hoàn toàn tự động, người dùng không cần nhập thủ công bất kỳ thông tin nào ngoài việc lựa chọn tệp PDF. Đây là ưu điểm nổi bật của hệ thống so với phương pháp nhập liệu truyền thống.

Trong quá trình thử nghiệm với nhiều báo cáo của các công ty chứng khoán khác nhau, hệ thống có thể xử lý ổn định các báo cáo có dung lượng từ vài trăm KB đến hàng chục MB. Các báo cáo có số lượng trang lớn vẫn được xử lý đầy đủ nhờ cơ chế đọc từng trang bằng PyPDF và gửi toàn bộ nội dung sang Gemini để phân tích.

3.4.2 Đánh giá các chức năng chính

Để đánh giá hiệu quả của hệ thống, từng chức năng được kiểm thử độc lập trước khi tích hợp thành hệ thống hoàn chỉnh.

Chức năng đọc PDF

Hệ thống có thể đọc chính xác phần lớn các báo cáo chứng khoán được phát hành dưới dạng PDF văn bản.

Quá trình thử nghiệm cho thấy nội dung văn bản được trích xuất đầy đủ, bao gồm các đoạn mô tả, bảng biểu dạng văn bản và các tiêu đề quan trọng. Điều này tạo tiền đề thuận lợi cho bước phân tích bằng mô hình AI.

Tuy nhiên, đối với các báo cáo được scan dưới dạng hình ảnh hoặc chứa nhiều biểu đồ mà không có lớp văn bản, PyPDF không thể nhận dạng được nội dung. Đây là hạn chế chung của phương pháp trích xuất văn bản trực tiếp từ PDF.

Chức năng phân tích bằng AI

Gemini có khả năng nhận diện chính xác hầu hết các trường dữ liệu quan trọng trong báo cáo như:

- Mã cổ phiếu.
- Công ty chứng khoán.
- Giá mục tiêu.
- Khuyến nghị đầu tư.
- Giá tại thời điểm phát hành.
- Luận điểm đầu tư.

Đặc biệt, mô hình có thể tổng hợp các đoạn phân tích dài thành các nội dung có cấu trúc mà vẫn giữ được ý nghĩa của báo cáo gốc.

Việc sử dụng Prompt Engineering kết hợp JSON Schema giúp dữ liệu đầu ra có định dạng ổn định hơn so với việc chỉ yêu cầu AI trả lời bằng văn bản tự do.

Chức năng lưu trữ dữ liệu

Sau khi AI trả kết quả, toàn bộ dữ liệu được lưu vào SQLite.

Trong quá trình kiểm thử, hệ thống ghi dữ liệu nhanh và không xảy ra lỗi mất dữ liệu.

Cơ chế kiểm tra trùng lặp trước khi thêm bản ghi mới giúp tránh việc lưu nhiều lần cùng một báo cáo.

Nhờ đó cơ sở dữ liệu luôn duy trì tính nhất quán.

Chức năng tra cứu

Việc tra cứu chỉ yêu cầu nhập mã cổ phiếu.

Sau khi truy vấn, hệ thống hiển thị:

- bảng định giá;
- giá mục tiêu;
- Upside;
- khuyến nghị đầu tư;
- toàn bộ phân tích chuyên sâu.

Việc tổng hợp nhiều báo cáo trên cùng một giao diện giúp người dùng dễ dàng so sánh quan điểm giữa các công ty chứng khoán.

Đây cũng là chức năng mang lại giá trị sử dụng lớn nhất của hệ thống.

3.4.3 So sánh với phương pháp thủ công

Nếu không sử dụng hệ thống, quy trình tổng hợp báo cáo thường bao gồm các bước sau:

1. Tìm kiếm báo cáo PDF.
2. Đọc toàn bộ báo cáo.
3. Ghi lại giá mục tiêu.
4. Ghi lại khuyến nghị.
5. Tóm tắt nội dung chính.
6. Lưu vào Excel.
7. So sánh giữa nhiều công ty chứng khoán.

Đối với một doanh nghiệp được năm công ty chứng khoán phân tích, người dùng có thể phải đọc hơn 150 trang tài liệu.

Trong khi đó, hệ thống chỉ yêu cầu tải các báo cáo lên một lần. Sau khi xử lý hoàn tất, toàn bộ dữ liệu đã được chuẩn hóa và lưu trữ, giúp việc tra cứu diễn ra gần như tức thời.

Bảng 3.1 trình bày sự khác biệt giữa hai phương pháp.

Tiêu chí	Phương pháp thủ công	Hệ thống đề tài
Đọc PDF	Thực hiện thủ công	Tự động
Trích xuất dữ liệu	Thủ công	Gemini
Chuẩn hóa dữ liệu	Không	Có
Lưu trữ	Excel hoặc Word	SQLite
So sánh báo cáo	Khó khăn	Trực tiếp
Tìm kiếm	Chậm	Theo mã cổ phiếu

Có thể thấy hệ thống giúp giảm đáng kể thời gian xử lý tài liệu và hạn chế sai sót do nhập liệu thủ công.

3.4.4 Ưu điểm của hệ thống

Qua quá trình triển khai và thử nghiệm, hệ thống có các ưu điểm sau:

- Kiến trúc đơn giản, rõ ràng và dễ mở rộng.
- Tự động hóa toàn bộ quá trình xử lý báo cáo.
- Không cần nhập dữ liệu thủ công.
- Có khả năng xử lý nhiều báo cáo của cùng một mã cổ phiếu.
- Chuẩn hóa dữ liệu bằng JSON Schema.
- Hỗ trợ loại bỏ dữ liệu trùng lặp.
- Giao diện thân thiện, dễ sử dụng.
- Dữ liệu được lưu trữ lâu dài trong SQLite.
- Có thể dễ dàng tích hợp với các mô hình AI khác trong tương lai.

Những ưu điểm này giúp hệ thống phù hợp với cả mục đích nghiên cứu lẫn xây dựng nguyên mẫu cho các ứng dụng phân tích tài chính.

3.4.5 Hạn chế

Bên cạnh những kết quả đạt được, hệ thống vẫn còn một số hạn chế.

Thứ nhất, chất lượng dữ liệu phụ thuộc nhiều vào khả năng trích xuất văn bản của PyPDF. Đối với các báo cáo scan hoặc chứa nhiều hình ảnh, lượng văn bản thu được có thể không đầy đủ, dẫn đến kết quả phân tích của AI bị ảnh hưởng.

Thứ hai, hệ thống phụ thuộc vào dịch vụ Gemini thông qua API. Khi kết nối Internet không ổn định hoặc vượt quá giới hạn số lượng yêu cầu, thời gian xử lý có thể tăng lên đáng kể.

Thứ ba, SQLite phù hợp với các ứng dụng cá nhân hoặc quy mô nhỏ nhưng chưa tối ưu khi có nhiều người dùng truy cập đồng thời.

Ngoài ra, hệ thống hiện mới hỗ trợ tiếng Việt và chủ yếu xử lý các báo cáo của thị trường chứng khoán Việt Nam. Đối với các báo cáo quốc tế có cấu trúc khác biệt, Prompt có thể cần được điều chỉnh để đạt kết quả tốt hơn.

3.4.6 Khả năng mở rộng

Nhờ kiến trúc module hóa, hệ thống có thể được phát triển theo nhiều hướng trong tương lai.

Một số hướng mở rộng bao gồm:

- thay SQLite bằng PostgreSQL hoặc MySQL để hỗ trợ nhiều người dùng;
- tích hợp thêm OCR nhằm xử lý các báo cáo scan;
- bổ sung cơ chế tìm kiếm theo công ty chứng khoán hoặc khoảng thời gian;
- xây dựng Dashboard trực quan bằng biểu đồ;
- tích hợp Retrieval-Augmented Generation (RAG) để trả lời câu hỏi trực tiếp từ dữ liệu đã lưu;
- hỗ trợ nhiều mô hình AI khác như GPT-5 hoặc Claude;
- xây dựng API để các hệ thống bên ngoài có thể khai thác dữ liệu.

Các hướng phát triển này sẽ giúp hệ thống tiến gần hơn tới một nền tảng quản lý và phân tích báo cáo chứng khoán hoàn chỉnh.

CHƯƠNG 4. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

4.1 Kết luận

Trong khuôn khổ học phần Project II, đề tài "**Xây dựng hệ thống tổng hợp báo cáo cổ phiếu**" đã hoàn thành các mục tiêu nghiên cứu và phát triển ban đầu. Hệ thống được xây dựng nhằm giải quyết bài toán tự động hóa quá trình khai thác thông tin từ các báo cáo phân tích chứng khoán, vốn trước đây chủ yếu được thực hiện thủ công và tiêu tốn nhiều thời gian.

Thông qua việc kết hợp các công nghệ hiện đại như PyPDF, Google Gemini API, SQLite và Streamlit, hệ thống đã xây dựng được một quy trình xử lý dữ liệu khép kín, bao gồm đọc báo cáo PDF, trích xuất nội dung văn bản, phân tích bằng mô hình ngôn ngữ lớn, chuẩn hóa dữ liệu theo định dạng JSON, lưu trữ vào cơ sở dữ liệu và cung cấp giao diện trực quan để người dùng tra cứu thông tin.

Kết quả thử nghiệm cho thấy hệ thống có thể xử lý ổn định nhiều báo cáo của các công ty chứng khoán khác nhau. Các thông tin quan trọng như mã cổ phiếu, công ty chứng khoán phát hành, ngày phát hành, khuyến nghị đầu tư, giá cổ phiếu tại thời điểm báo cáo và giá mục tiêu được trích xuất với độ chính xác cao. Đồng thời, các nội dung phân tích chuyên sâu như tình hình hoạt động doanh nghiệp, triển vọng kinh doanh, dự báo tài chính, các yếu tố rủi ro và luận điểm đầu tư cũng được tổng hợp và lưu trữ thành công.

Một điểm nổi bật của hệ thống là việc sử dụng Prompt Engineering kết hợp với JSON Schema để kiểm soát đầu ra của mô hình Gemini. Cách tiếp cận này giúp dữ

liệu thu được có cấu trúc thống nhất, hạn chế các lỗi định dạng thường gặp khi làm việc với các mô hình ngôn ngữ lớn và tạo thuận lợi cho việc lưu trữ cũng như truy vấn dữ liệu.

Ngoài ra, cơ chế kiểm tra dữ liệu trùng lặp trước khi lưu vào cơ sở dữ liệu góp phần đảm bảo tính nhất quán của hệ thống. Giao diện được xây dựng bằng Streamlit có cách sử dụng đơn giản, cho phép người dùng tải báo cáo, tra cứu thông tin và quản lý dữ liệu mà không cần thao tác trực tiếp với cơ sở dữ liệu.

Trong quá trình thực hiện đề tài, sinh viên đã có cơ hội vận dụng tổng hợp nhiều kiến thức đã học như lập trình Python, phát triển ứng dụng Web, thiết kế cơ sở dữ liệu, xử lý tài liệu PDF, tích hợp API của mô hình ngôn ngữ lớn và xây dựng quy trình xử lý dữ liệu hoàn chỉnh. Bên cạnh đó, đề tài cũng giúp tiếp cận các kỹ thuật Prompt Engineering và khai thác hiệu quả các mô hình AI hiện đại trong một bài toán có tính ứng dụng thực tế.

Mặc dù vẫn còn một số hạn chế nhất định, kết quả đạt được cho thấy hệ thống đáp ứng tốt các mục tiêu đã đề ra và có khả năng phát triển thành một nền tảng hỗ trợ phân tích tài chính trong tương lai.

4.2 Hướng phát triển

Mặc dù đã đạt được những kết quả khả quan, hệ thống vẫn còn nhiều tiềm năng để tiếp tục nghiên cứu và mở rộng nhằm đáp ứng tốt hơn nhu cầu của người sử dụng.

Hướng phát triển đầu tiên là nâng cao khả năng xử lý tài liệu. Hiện nay hệ thống chủ yếu làm việc với các báo cáo PDF có chứa văn bản số. Đối với các tài liệu được quét dưới dạng hình ảnh hoặc có chất lượng thấp, có thể tích hợp thêm công nghệ nhận dạng ký tự quang học (OCR) như Tesseract OCR hoặc Google Cloud Vision để tăng khả năng trích xuất dữ liệu.

Hướng phát triển thứ hai là mở rộng cơ sở dữ liệu. Trong phiên bản hiện tại, hệ thống lưu trữ các thông tin cơ bản như giá mục tiêu, khuyến nghị và các nội dung phân tích chính. Trong tương lai, có thể bổ sung thêm nhiều chỉ tiêu tài chính quan trọng như EPS, BVPS, ROE, ROA, P/E, P/B, EV/EBITDA, tốc độ tăng trưởng doanh thu, lợi nhuận sau thuế và dòng tiền. Việc mở rộng này sẽ giúp người dùng có thêm cơ sở để so sánh và đánh giá doanh nghiệp.

Bên cạnh đó, thay thế SQLite bằng các hệ quản trị cơ sở dữ liệu mạnh hơn như PostgreSQL hoặc MySQL sẽ giúp hệ thống đáp ứng tốt hơn khi số lượng báo cáo và người dùng tăng lên. Đồng thời, việc triển khai trên máy chủ sẽ tạo điều kiện để nhiều người dùng có thể truy cập và khai thác dữ liệu đồng thời.

Một hướng phát triển quan trọng khác là xây dựng hệ thống hỏi đáp thông minh (Question Answering System) dựa trên cơ sở dữ liệu đã lưu. Thay vì chỉ tìm kiếm theo mã cổ phiếu, người dùng có thể đặt các câu hỏi bằng ngôn ngữ tự nhiên như:

- "Doanh nghiệp nào được nhiều công ty chứng khoán khuyến nghị mua nhất?"
- "Các rủi ro chính của cổ phiếu FPT là gì?"
- "So sánh giá mục tiêu của HPG giữa SSI và VNDIRECT."

Để thực hiện được điều này, có thể kết hợp kỹ thuật Retrieval-Augmented Generation (RAG) nhằm truy xuất các báo cáo liên quan trước khi gửi ngữ cảnh đến mô hình ngôn ngữ lớn, giúp câu trả lời chính xác và bám sát dữ liệu hơn.

Ngoài ra, hệ thống cũng có thể được mở rộng theo hướng trực quan hóa dữ liệu bằng cách bổ sung các biểu đồ thống kê về giá mục tiêu, tỷ lệ Upside, phân bố khuyến nghị đầu tư hoặc sự thay đổi quan điểm của các công ty chứng khoán theo thời gian. Các biểu đồ này sẽ giúp người dùng nhanh chóng nắm bắt xu hướng chung mà không cần đọc từng báo cáo riêng lẻ.

Một hướng phát triển khác là tự động cập nhật dữ liệu từ các nguồn công khai hoặc trang thông tin của các công ty chứng khoán. Khi có báo cáo mới được phát hành, hệ thống có thể tự động tải về, phân tích và bổ sung vào cơ sở dữ liệu mà không cần người dùng thao tác thủ công.

Cuối cùng, việc triển khai hệ thống trên nền tảng điện toán đám mây sẽ giúp mở rộng khả năng phục vụ nhiều người dùng, đồng thời tạo tiền đề phát triển thành một nền tảng hỗ trợ nghiên cứu và phân tích chứng khoán trực tuyến. Khi kết hợp với các mô hình AI ngày càng mạnh hơn trong tương lai, hệ thống không chỉ dừng lại ở việc tổng hợp báo cáo mà còn có thể hỗ trợ phân tích xu hướng, đánh giá doanh nghiệp và đưa ra các gợi ý đầu tư dựa trên dữ liệu lịch sử.